Exam in Concurrent Algorithms and Data Structures

Department of Information Technology
Uppsala University
2025–01-09
Lecturer: Parosh Aziz Abdulla
Location: Danmarksgatan 30
Time: 8:00 - 13:00

No books or lecture notes allowed
The use of mobile phones, calculators, and computers is not allowed
Directions:

1. If a problem requires a *yes* or *no* answer, please start your answer with one of the words *yes* or *no*. Otherwise, you will not get any credit for the answer.

2. All explanations should be concise and to the point. Lengthy explanations might yield a lower grade (and will not give extra points).

Good Luck!

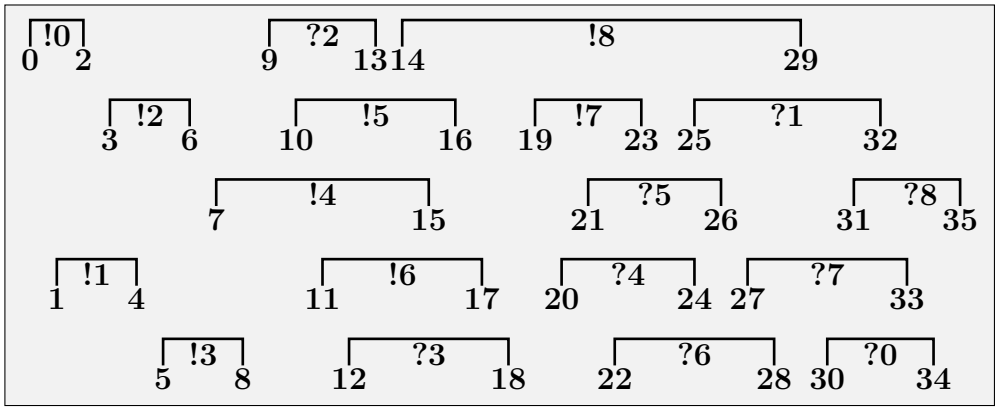

Figure 1: A stack history. Here, ! is a push operation and ? is a pop operation.

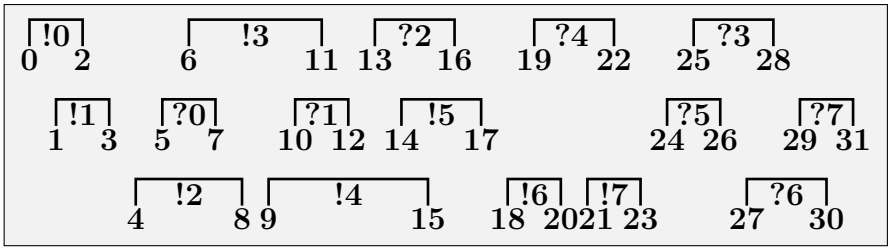

Figure 2: A queue history. Here, ! is an enqueue operation and ? is a dequeue operation.

Problem 1 [**Histories**] For the concurrent histories in Fig. 1, answer whether the given history is linearizable for the given abstract data type. If *yes* add the linearization points in the Figures. If *no* explain the reason. For each case, assume that the given history is initiated from the empty stack/queue.

```
add(k):
Node p, c
1  lock(H)
2  p = H
3  c = p.next
4  lock(c)
5  while (c.key < k)
6    unlock(p)  unlock(c)
7    p = c
8    c = c.next
9    lock(c)
10 if (c.key = k)
11   return false
12 else
13   n = new Node(k,-)
14   n.next = c
15   p.next = n
16   return true
17 unlock(c)
18 unlock(p)
```

```
rmv(k):
Node p, c
1  lock(H)
2  p = H
3  c = p.next
4  lock(c)
5  while (c.key < k)
6    unlock(p)
7    p = c
8    c = c.next
9    lock(c)
10 if (c.key = k)
11   p.next=c.next
12   return true
13 else
14   return false
15 unlock(c)
16 unlock(p)
```

Figure 3: The modified Fine-Grained Synchronization Algorithm.

Problem 2 [**Fine-Grained Synchronization**]
Assume that we change the Fine-grained locking concurrent set algorithm so that the code for the `add` method locks only one cell during traversal. The new algorithm is depicted in Fig. 3. As usual, the algorithm uses sentinel nodes to mark the left and right ends of the list. Notice that we have removed Line 1 and Line 18 and replaced Line 6 with a new line where we unlock c instead of p. The `rmv` method nevertheless implements the "hand-over-hand" protocol. Will the algorithm still be linearizable? If *yes*, describe a correct linearization policy. If *no*, give a counter-example in the form of a concurrent history H such that (i) H can be generated by the program, and (ii) H is not linearizable.
Remark We do not consider the `ctn` operation in this problem.

Problem 3 [**Two-Counter Abstract Data Type**] Consider a *two-counter* abstract data type, called 2.Counter, operating on two counters c and d. The ADT allows the following operations:

- `inc`(c) and `inc`(d) respectively increment the counters' values by one.
- `dec`(c) and `dec`(d) respectively decrement the counters' values by one.
- `check`() that returns one of two values, namely
  - equal, if the current values of the counters are equal.
  - not.equal, if the current values of the counters are not equal.
- Give the abstract data type 2.Counter.

Next, assume we design a library that implements 2.Counter. The library allows n threads to operate concurrently. We refer to the threads as $\theta_1, \ldots, \theta_n$. The thread θ_1 operates on two registers representing the two counters, namely $\mathtt{R}_{1,1}$ and $\mathtt{R}_{1,2}$. Similarly, the thread θ_2 operates on the registers $\mathtt{R}_{2,1}$ and $\mathtt{R}_{2,2}$. In general, the thread θ_i operates the registers $\mathtt{R}_{i,1}$ and $\mathtt{R}_{i,2}$. Intuively, the current value of the counter c is the sum of the values stored in the registers $\mathtt{R}_{1,1}, \mathtt{R}_{2,1}, \ldots, \mathtt{R}_{n,1}$, the current value of the counter d is the sum of the values stored in the registers $\mathtt{R}_{1,2}, \mathtt{R}_{2,2}, \ldots, \mathtt{R}_{n,2}$. The registers can be *read* by all threads; however, only thread θ_i can *write* to register $\mathtt{R}_{i,1}$ and $\mathtt{R}_{i,2}$. A register may carry multiple values, such as two integers or an integer and a Boolean, and so on. Assume that the registers are atomic, i.e., calls to the registers cannot overlap.

- Write the pseudo-codes for the three operations.

  Remark. You are not allowed to use locks or CAS statements. Your code should only use the mentioned registers.

- Describe the linearization policy to justify the correctness of your code.

Problem 4 [**Semi-Coarse Set**] We want to design a concurrent set library

Algorithm 1: add(k).

```
acquired := 0
while not acquired do
    acquired := CAS (lock, 0, 1)
    if acquired then
        p := H
        c := p · next
        while c · key < k do
            p := c
            c := c · next
        if c · key = k then
            return 0
        else
            n := new Node (k, −)
            n · next := c
            p · next := n
            return 1
lock := 0
exit
```

using Algorithm 1, Algorithm 2, and Algorithm 3 to implement the three set operations respectively. We use a global lock which is a Boolean shared variable lock, which we initialize to 0. As usual, the set is stored in a singly linked list, in which each cell stores an integer key and a pointer to the next cell. The list has two sentinel cells: the left-most cell stores the key $-\infty$, and it is pointed to by H; and the right-most cell stores the key ∞, and it is pointed to by T.

Let us consider the code for the add operation. The algorithm executes its code repeatedly until it succeeds in acquiring the lock. We indicate this using the local Boolean variable acquired, which we assign 1 only when the thread has acquired the lock. The thread acquires the lock using a CAS in which it atomically (i) tests whether lock is 0, and (ii) sets lock to 1. It then performs the standard search procedure to check whether the input k is already stored in the list. If *yes*, it returns 0; if *no*, it inserts a new cell containing k as key and returns 1. Notice that the return statement does not exit the procedure.

Algorithm 2: rmv(k).

```
acquired := 0
while not acquired do
    acquired := CAS (lock, 0, 1)
    if acquired then
        p := H
        c := p · next
        while c · key < k do
            p := c
            c := c · next
        if c · key = k then
            return 0
        else
            p · next := c · next
            return 1
lock := 0
exit
```

In both cases above, the thread returns the lock by setting lock to 0 and then exits.
We can explain the rmv(k) procedure similarly to the add(k) procedure. The ctn procedure does not acquire (or release any locks).

- Is the library linearizable concerning the abstract data type SET?
  - If *yes*, give the linearization policy.
  - If *no*, give a non-linearizable history.

Algorithm 3: The ctn operation.

```
c := H
while c · key < k do
    c := c · next
if c · key > k then
    return 0
else
    return 1
exit
```

```
push(k):
Node t
n = new Node(k,-)
while (true)
  t = Top
  n.next = t
  if (CAS (Top,t,n))
    exit
```

(a) The push operation in the Treiber stack.

```
pop:
Node t
while (true)
  t = Top
  if (t = NULL)
    return *
    exit
  if (CAS (Top,t,t.next))
    return t.val
    exit
```

(b) The pop operation in the Treiber stack.

Problem 5 Recall the Treiber stack algorithm, depicted in Fig. 4a and Fig. 4b. We would like to extend the stack abstract data type by adding a third operation, count, that calculates the number of elements currently inside the stack.

- Define the abstract data type for the extended stack variant.
- Define the code for the count operation.
- Give the linearization policy for all three operations. Motivate why it is correct.